

Graph Algorithm exam

Time: 1 hour 45 minutes

Problem 1. – 2p

Consider the following algorithm:

Algorithm name: Vertex Cover by heuristic

Input:

g: an undirected graph

Output:

vertex_cover: a set containing the vertices that form a vertex cover

Algorithm:

```
Set vertex_cover
while g.get_e() > 0:
    vertices = g.get_vertices()
    max = vertices[0]
    max_neighbors = g.neighbors(max)
    for i from 1 to v-1:
        new_neighbors = g.neighbors(vertices[i])
        if len(new_neighbors) > len(max_neighbors):
            max = vertices[i]
            max_neighbors = new_neighbors
    end_if
end_for
vertex_cover.add(max)
for neighbor in max_neighbors:
    g.remove_edge(max, neighbor)
end_for
end_while
return vertex_cover
end_algorithm
```

Analyze the time complexity of the algorithm in the following cases:

- a) g is implemented using a list of neighbors
- b) g is implemented using an adjacency matrix

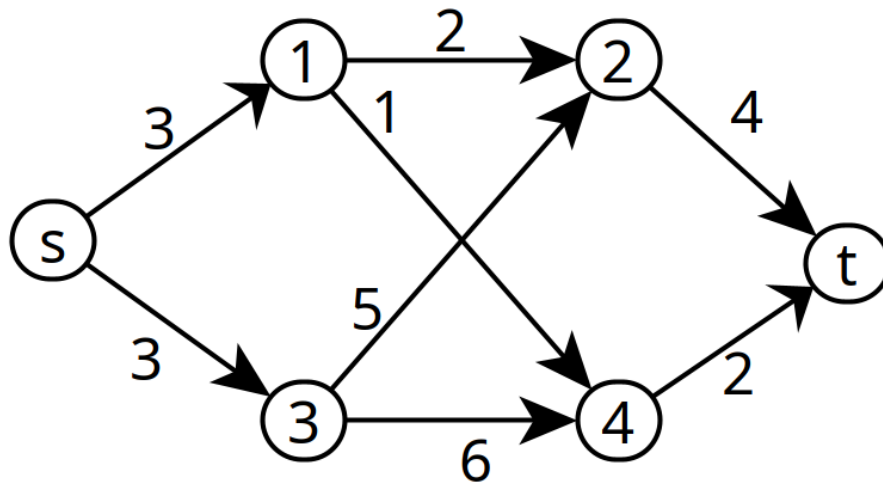
You have to properly justify the complexity of the algorithm simply writing that the algorithm is in some complexity will not be considered an answer.

Notes: - you can consider that “get_vertices” is in $\Theta(v)$ in both cases.

- you can use “c” as a variable for the number of vertices in the vertex cover

Problem 2. – 3p

Consider the following flow network (the number on the edges represent the capacity of the edge):

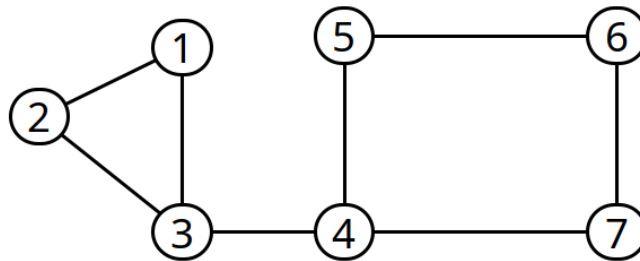


Exercises:

- 1) Give the values of a flow f through this network such that $|f| = 4$. (0.5 p)
- 2) What is the maximum possible total flow for this network ($|f|_{\max}$)? (0.25 p)
- 3) What is the cut with minimum capacity? (0.5p)
- 4) Draw the residual network for this network flow and the flow f you have given at 1). (0.75p)
- 5) Explain how the residual network can be used to find a flow that has a higher total flow. Use the method to find a different flow for the example. (1p)

Problem 3 – 4p

In a graph a **bridge** is an edge that connects 2 parts of the graph that would be disconnected without the bridge. In other words, if the graph is connected, if you remove a bridge, the graph will become disconnected. In the following example, edge (3,4) is a bridge.



Create an algorithm that, given a connected, undirected graph, returns a list with all the bridges in the graph (a list with all the edges that are a “bridge”).

You must implement the algorithm in at most $O((v+e)^2)$. You must explain why your algorithm respects this bound.

Write the algorithm either in Python or pseudocode – same style of pseudocode as in the lecture notes.

Important: the input graph only has implemented the methods described below. You must use the input graph with those methods, you do not know how the graph is implemented (e.g. if it is an adjacency matrix or list of neighbors), you cannot access the internal structure of the graph and you cannot create new methods for the graph. When solving the problem you **must** use only these methods as described (it is not required to use all of them).

It is allowed to create auxiliary algorithms or functions. Also, you are allowed to use other data structures (list, stack, queue, set, map etc.) and you can consider that they are implemented efficiently.

Read these before starting on problem 3. The graph has the following methods:

- **get_vertices()** – returns a list with the vertices of the graph – $\Theta(v)$
- **get_edges()** – returns a list with the edges of the graph, each edge being a pair of 2 vertices (e.g. the list of edges might be [(1,2), (3,5), (3,2)])
– $\Theta(e + v)$

- **get_v()** – returns the number of vertices (v) – $\Theta(1)$
- **get_e()** – returns the number of edges (e) – $\Theta(1)$
- **add_edge(v1, v2)** – adds the edge ($v1, v2$) to the graph – $\Theta(1)$
- **remove_edge(v1, v2)** – removes the edge ($v1, v2$) from the graph – $O(e/v)$
- **neighbors(v)** – returns a list of the neighbors of vertex v . The list contains the other vertices (e.g. if the edges are $[(1,2), (3,5), (3,2)]$ then `get_neighbors(3)` will return $[5,2]$) – $O(e/v)$

Default – 1p